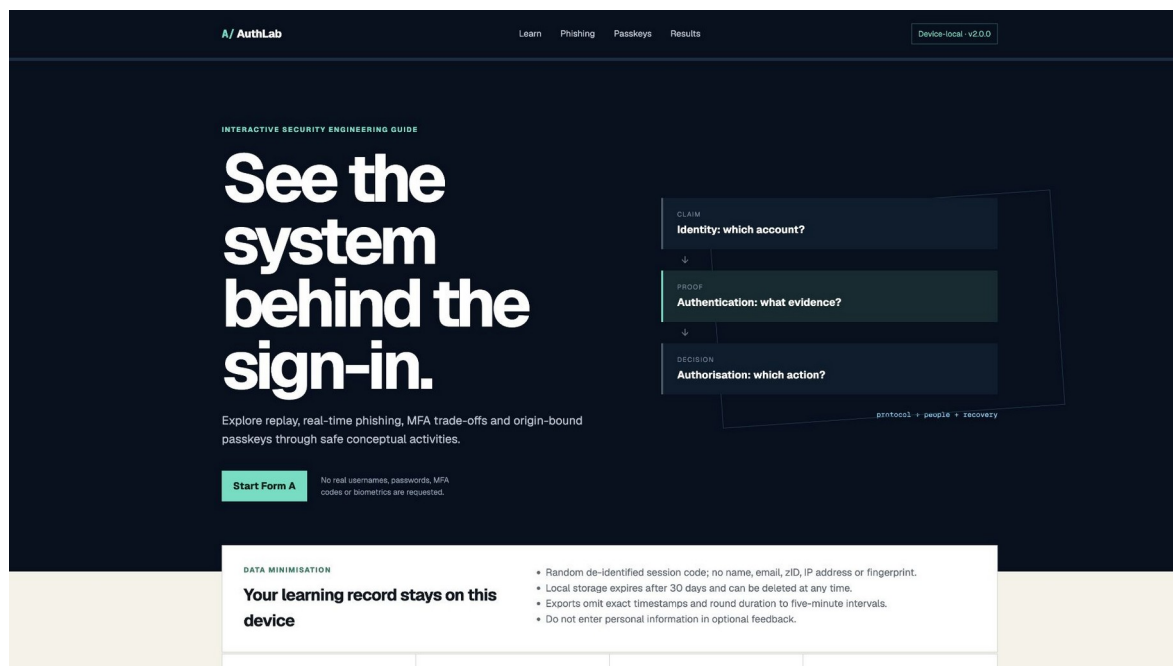


SECURITY ENGINEERING PROJECT REPORT

From Passwords to Passkeys

An interactive guide to authentication, phishing, MFA and WebAuthn



Course COMP6441 Security Engineering

Student Name: _____ zID: _____

Audience University students and cybersecurity beginners

Prepared 20 July 2026

Evidence boundary

The final artefacts and technical checks are documented. No participant result, work hour, personal experience or reflection has been generated. Human teaching effectiveness remains unevaluated until genuine consented records are supplied.

Abstract

AuthLab is a browser-based learning resource for beginners who need to reason about authentication as a security system rather than a list of password tips. Four interactive modules separate identity, authentication and authorisation; model credential stuffing with fictional data; require evidence-based phishing decisions; and compare password, OTP, push and passkey mechanisms under an explicit attacker model. Parallel six-item Form A and Form B tests map to stable learning objectives. Form A records answers without revealing correctness; Form B then supplies the correct answer, rationale, major-distractor explanations and objective mapping. Progress is stored locally, expires after 30 days and can be deleted by the learner. Export is a de-identified learning record with no name, email, exact timestamp, IP address, browser fingerprint, real credential or biometric. Automated tests and responsive browser checks verify implemented behaviour; they do not demonstrate learner improvement. No genuine participant dataset was found, so the teaching-effectiveness result is explicitly 'not yet evaluated'.

1. Introduction and problem statement

Beginners often treat a login as one event and 'MFA' as one security level. That hides the engineering boundaries that matter: the claimed identity may be wrong, the accepted proof may be replayable or phishable, and a valid session may still receive excessive authority. AuthLab asks a narrower research question: how can a beginner-facing resource make those boundaries and attack paths visible without collecting credentials or normalising unsafe behaviour?

Project goal

Enable a learner to identify the failed layer, choose a control that changes the attack path, and state the residual risk precisely.

2. Project scope and learning objectives

The project category is Teach Something. The scope is a self-contained static web activity plus learner workbook, facilitator guide, presentation, short narrated demonstration and evaluation tooling. It is not a password manager, identity provider, production relying party, penetration-testing tool or claims-based research study.

ID	Learning objective
LO-01	Distinguish identity, authentication and authorisation in one sign-in flow.
LO-02	Explain how one leaked and reused password enables cross-service credential stuffing.
LO-03	Use registered-domain, origin, context and trusted-route evidence in phishing decisions.
LO-04	Distinguish replay resistance from phishing resistance.
LO-05	Explain what the relying party stores and what the authenticator protects in WebAuthn.
LO-06	Explain verifier-name/RP-ID binding and name at least one residual passkey risk.

2.1 Design constraints

- No real password, OTP, recovery answer, production account or biometric input.
- No analytics, remote lesson-record endpoint or identity field.
- Fictional services and reserved example domains only.
- Conceptual passkey ceremony is the default; the browser API demonstration is optional.
- Technical verification, source validation and human learning evidence are reported separately.

3. Background and literature

3.1 Replay and credential stuffing

A reusable password exposed at one service can be tried against unrelated services. OWASP describes credential stuffing as automated use of stolen username/password pairs. The second service need not have suffered its own database breach: it fails because the same verifier accepts the same reusable proof. NIST SP 800-63B-4 therefore treats password blocklists, rate limiting and password-manager support as system controls; complexity rules alone do not remove reuse or phishing risk.

3.2 Phishing resistance

NIST distinguishes replay resistance from phishing resistance. A short-lived TOTP can reduce an ordinary replay window yet still be relayed in real time through an adversary-in-the-middle site. Phishing resistance requires a protocol property such as verifier-name binding, rather than an expectation that the user will always recognise copied visual design.

3.3 Passkeys and WebAuthn

A WebAuthn credential is scoped to a relying party. The relying party stores public-key material and credential metadata; the authenticator protects the private key. Authentication signs a fresh challenge, and a production server verifies the challenge, origin, RP ID binding, flags and signature. The private key is not sent to the relying party during authentication. Syncable passkeys may synchronise protected key material within a platform ecosystem, so ‘never leaves one device’ is not a universal claim.

Document status

Web Authentication: An API for accessing Public Key Credentials — Level 3 is a W3C Candidate Recommendation Snapshot dated 26 May 2026, not a final W3C Recommendation.

4. Threat model

Assume an attacker possesses a leaked password and may operate a polished real-time adversary-in-the-middle phishing site. The attacker may relay passwords, TOTP codes or approval requests to the legitimate service. The comparison does not assume a fully compromised learner device, authenticator, browser or relying-party server; those remain residual risks. Account recovery is analysed separately because a weak recovery route can bypass a strong primary authenticator.

Mechanism	Replay	Phishing	Concise rationale / residual risk
Password only	No resistance	No resistance	Reusable shared proof; reuse, disclosure and guessing remain.
Password + SMS OTP	Limited	No resistance	Short-lived code may be relayed; telecom and recovery risks remain.
Password + TOTP	Short-lived	No resistance	A live phishing proxy can relay a valid code before expiry.
Password + push	Often better	Not inherent	Fatigue and weak transaction context can cause false approval.
Passkey / WebAuthn	Resistant	Resistant	Fresh challenge and RP binding; device, recovery and service logic still matter.

4.1 Safety risks created by the teaching artefact

The project could itself cause harm if it asked for real credentials, copied brands, uploaded identifiable learning records or described a client-only API call as a secure login. Controls include dummy data, reserved domains, local-only state, a 30-day expiry, explicit deletion, de-identified export, safe CSV cells, informed WebAuthn consent and repeated production-limit wording.

5. Methodology and instructional design

The sequence moves from familiar decisions to protocol properties. Module 1 gives a vocabulary and a checkpoint on excessive privilege. Module 2 changes one variable—proof reuse—inside a harmless fixed simulation. Module 3 withholds scenario feedback until a decision is made, then explains the decisive domain or trusted route. Module 4 first reveals a five-step conceptual passkey ceremony and only then offers an optional real browser API call. Opening a section never marks it complete; each module requires an explicit checkpoint.

5.1 Parallel assessment design

Form A and Form B contain different wording and scenarios but map one-to-one to LO-01 through LO-06 at comparable conceptual difficulty. Form A confirms that answers were recorded and withholds scores, correctness and rationales until Form B is submitted. This reduces direct answer rehearsal. Form B reveals the correct answer, a short rationale, major-distractor explanations and the related objective. Pre- and post-confidence are labelled subjective and kept separate from knowledge scores.

A/ AuthLab Learn Phishing Passkeys Results Device-local v2.0.0

FORM B FEEDBACK

Review the evidence behind every answer

CORRECT · ITEM 1 · LO-01

A valid user proves control but can edit every account. Which layer failed?

Your answer: Authorisation

Correct answer: Authorisation

Why: The proof was accepted, but the permitted action was too broad: that is an authorisation failure.

▼ **Why the other options are weaker**

- **Identity:** The account identity can be correct while permissions are excessive.
- **Authentication:** Authentication succeeded in the scenario.

LO-01: Distinguish identity, authentication and authorisation in a sign-in flow.

CORRECT · ITEM 2 · LO-02

Service A leaks a password that is also used at Service B. What makes Service B vulnerable?

Figure 1. Form B review displays textual correctness, rationale, distractor explanations and the mapped objective.

6. Implementation

AuthLab uses React/TypeScript with a vnext production build and a Worker entry point. The content and UI live in app/page.tsx; versioned scoring, persistence, export and CSV safety live in lib/authlab-model.ts. A schema-version check safely resets incompatible or corrupt local data. Duration is rounded to five-minute intervals. JSON/CSV export is initiated by the learner and contains stable question, objective and scenario IDs. A formula-prefixed CSV value is neutralised before RFC-style quoting, including commas, quotation marks, newlines and Unicode.

Component	Security responsibility	Evidence
Parallel forms	Prevent immediate pre-answer revelation	model.test.ts; Form B screenshot
Local session	Validate version, expire, reset and delete	authlab-model.ts; storage tests
Export	Minimise and de-identify learner records	schema docs; JSON/CSV tests
CSV encoder	Prevent spreadsheet formula execution	punctuation/formula edge-case tests
Worker headers	CSP, referrer, nosniff, anti-frame, permissions	rendered response tests
WebAuthn demo	Explicit consent and no persistent credential material	source; state tests; disclosure screenshot

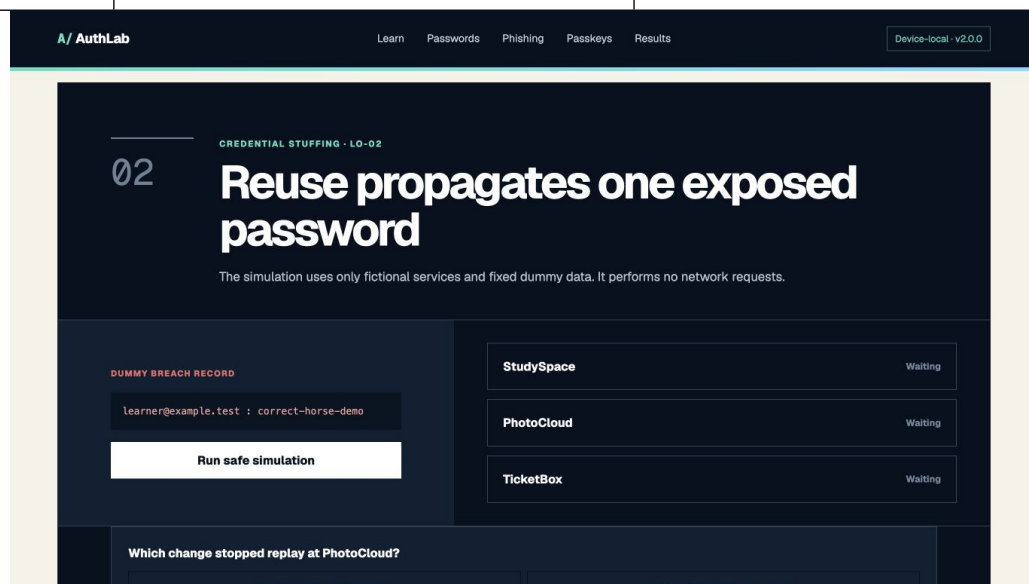


Figure 2. The credential-stuffing module uses fixed fictional data and performs no network request.

7. WebAuthn implementation boundary

The default activity reveals a conceptual ceremony without invoking an authenticator. An optional disclosure states that a test credential may be created for the current origin, that the browser/authenticator handles local user verification, that the site does not receive biometric data, and that cancellation is safe. Explicit consent is required. The create request uses a cryptographically generated challenge and residentKey: 'discouraged'. The credential is held only in component memory so that an optional get request can demonstrate an assertion request.

This is not a production authentication system. There is no server-side challenge store, registered credential database, account binding, counter policy, assertion-signature verification, replay detection or recovery design. The interface exposes only non-secret status/identifier metadata and never exports credential material. A client-only create/get sequence is useful for observing API boundaries, but it cannot prove a secure login.

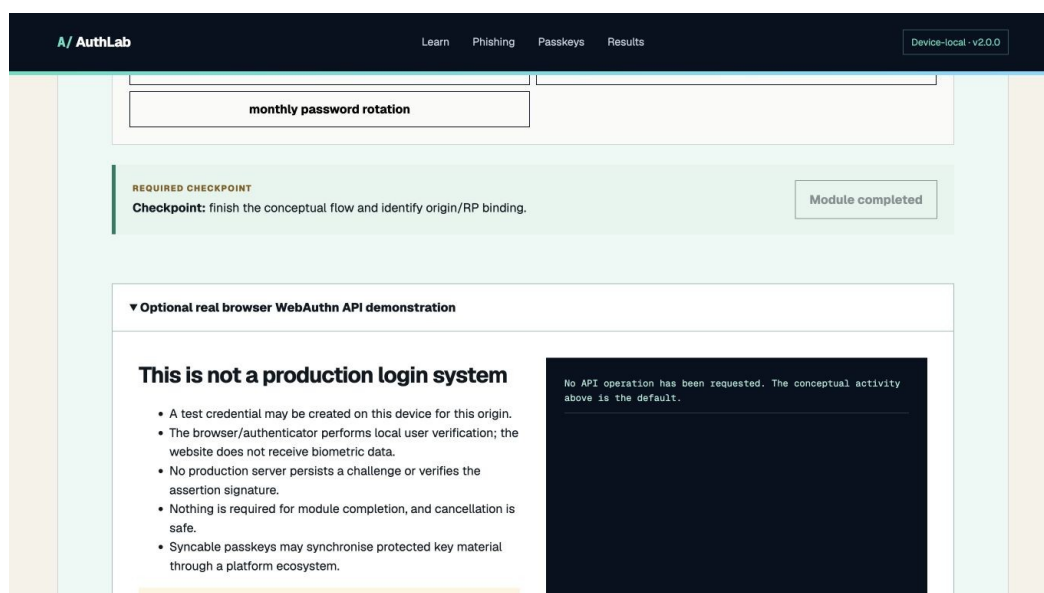


Figure 3. Optional API demonstration with explicit limitations and consent; no API operation has been requested in this evidence capture.

8. Technical verification

Final command outputs are preserved under evidence/logs. Behavioural tests cover Form A/Form B scoring and mapping, hidden pre-test feedback, post-test rationales, item/scenario/confidence export, CSV edge cases, corrupt/expired/version-mismatched storage, deletion and WebAuthn unsupported/cancel/timeout states. Rendered tests inspect the production shell and actual Worker response headers. Source checks also confirm the absence of a password input and outbound fetch call.

Verification	Status	What it establishes
npm ci	Pass — see log	Lockfile installation succeeds.
npm run lint	Pass — see log	Final source passes configured lint checks.
npm test	Pass — see log	Model and rendered-response checks pass.
npm run build	Pass — see log	Production artefact compiles.
npm audit --omit=dev	See final log	Advisory scan result, not proof of absence of flaws.
Responsive browser QA	Pass with noted limits	No horizontal overflow at four required viewports.
Human learning evaluation	Not run	No effectiveness conclusion is available.

Interpretation

The recorded 6/6 Form A and 5/6 Form B developer path deliberately selected one wrong TOTP answer. It verifies hidden/revealed feedback, score change and export behaviour; it is not a participant result or learning measure.

9. Human evaluation and results

No genuine participant export, consent record or facilitator observation was present. The human-evaluation status is therefore 'not run', and the result is 'no claim'. The package provides participant information, consent text, a facilitator protocol, a versioned export schema, a blank CSV template and an analysis script. These are evaluation infrastructure, not evaluation evidence.

9.1 Planned descriptive analysis

1. Report the number of valid de-identified records and all rejected/malformed records.
2. Show each Form A/Form B score and difference, including zero or negative differences.
3. Report mean and median descriptively for a small convenience sample; do not claim statistical significance.
4. Inspect item- and learning-objective-level correctness, subjective confidence change and scenario decisions.
5. Code optional feedback into an auditable keyword/theme worksheet without treating frequency as causal proof.

9.2 Threats to validity

Any later small study will be limited by convenience sampling, self-selection, practice effects between parallel forms, no delayed retention test, facilitator influence, browser/authenticator variation, simplified phishing scenarios and subjective confidence ratings. A short intervention cannot establish durable real-world behaviour.

10. Discussion and design changes

The strongest design change was treating evaluation validity as a security property of the learning system. An identical pre/post test with immediate answer revelation would produce contaminated evidence, even if the interface looked polished. Separating parallel forms and delaying feedback makes the result more interpretable. The same principle changed the export from aggregate scores to versioned item-level records while reducing identifying detail.

A second change was narrowing the passkey claim. 'Phishing-resistant' is supported only under the WebAuthn threat model and does not mean invulnerable. Recovery, device compromise, browser/authenticator implementation, ecosystem governance, malicious relying-party logic and excessive authorisation remain. The optional API call was therefore demoted beneath a conceptual default and labelled as non-production.

10.1 Challenges

Observed contradiction	Engineering response	Evidence
Pre-test exposed answers	Parallel forms; reveal only after Form B	Model tests; screenshot
'Anonymous' overclaim	Use 'de-identified'; remove exact timestamps/identity	Schema and export tests
Client API presented like login	Conceptual default; consent; server-limit notice	Disclosure and source
Ambiguous High/Medium/Low matrix	Explicit attacker assumptions and rationales	Module 4
Starter code widened attack surface	Remove auth/database examples and dependencies	README; package manifest

11. Ethical and professional considerations

The artefact demonstrates concepts without requesting production secrets or targeting real brands. Data minimisation is enforced in code as well as described in policy. Learners can inspect, export and delete their device-local record. The consent text warns against personal information in feedback. Public deployment makes the teaching pages accessible to anyone with the URL, but does not make local records public or transmit them to the host.

12. Personal reflection

Student-authored section required

This report does not invent a personal reflection. Before submission, the student should replace this box with their own evidenced account: What did I originally misunderstand? What specific failure or contradiction exposed it? Which file, test or feedback proves that? What did I change? How did this alter my security-engineering understanding?

13. Limitations

The site has no verified participant-learning dataset, delayed retention result, screen-reader user study or comprehensive cross-browser/authenticator matrix. The optional WebAuthn path lacks production server-side verification and account recovery. Phishing scenarios are simplified. Confidence is subjective. The public site depends on the availability and policies of the hosting platform. These limits constrain claims about effectiveness and operational security, but they are made visible rather than hidden.

14. Conclusion

AuthLab turns authentication from a set of slogans into inspectable decisions, protocol properties and residual risks. The finished engineering artefact supports a safe beginner journey, valid parallel assessment mechanics, minimised local evidence and precise WebAuthn boundaries. Technical checks establish that the artefact behaves as documented. Whether it improves learning remains a separate empirical question that requires genuine consented participants.

References

- [1]. *Digital Identity Guidelines: Authentication and Authenticator Management (NIST SP 800-63B-4)*. National Institute of Standards and Technology, final, July 2025; especially §§3.2.5 and 3.2.7; accessed 20 July 2026. <https://pages.nist.gov/800-63-4/sp800-63b.html>
- [2]. *Web Authentication: An API for accessing Public Key Credentials — Level 3*. World Wide Web Consortium, Candidate Recommendation Snapshot, 26 May 2026; accessed 20 July 2026. <https://www.w3.org/TR/webauthn-3/>
- [3]. *Passkeys*. FIDO Alliance, official overview; publication date not shown; accessed 20 July 2026. <https://fidoalliance.org/passkeys/>
- [4]. *Credential Stuffing*. OWASP Foundation, community attack reference; publication date not shown; accessed 20 July 2026. https://owasp.org/www-community/attacks/Credential_stuffing
- [5]. *Phishing Guidance: Stopping the Attack Cycle at Phase One*. CISA, NSA, FBI and MS-ISAC, March 2025; accessed 20 July 2026. <https://www.cisa.gov/sites/default/files/2025-03/Phishing%20Guidance%20-%20Stopping%20the%20Attack%20Cycle%20at%20Phase%20One%20508.pdf>

Generative AI acknowledgement

OpenAI Codex assisted with code review, refactoring, test generation, primary-source checking, document/slide formatting, language editing and quality assurance. Security claims were checked against the primary sources listed above. The submitting student must review, understand and accept responsibility for the final work and retain the course-approved disclosure. Codex did not generate participant data, work hours, personal reflection or a student identity.

Appendix A — Proposal-to-evidence traceability

Proposal objective	Implemented feature	Technical / evaluation evidence	Status
Explain identity, authentication and authorisation	Module 1 + LO-01 parallel questions	module-1 screenshot; model tests; A/B items	Complete technically
Demonstrate password reuse and credential stuffing	Fictional fixed-data simulation + checkpoint	module-2 screenshot; scenario export	Complete technically
Teach phishing recognition and safer action	Four domain/context decisions + feedback	module-3 screenshot; scenario records	Complete technically
Compare MFA mechanisms	Threat model and five-row matrix	Module 4; Form A/B LO-04	Complete technically
Explain passkey/WebAuthn flow	Conceptual ceremony + optional consented API demo	module-4/disclosure screenshots; state tests	Complete technically
Evaluate teaching effectiveness	Parallel consent/protocol/export/analysed forms,	No genuine participant exports	Prepared; not run

Appendix B — Required student actions

- Replace [STUDENT_NAME] and [ZID], then rename the final report to Report-[ZID].pdf.
- Write and defend the personal-reflection section using genuine evidence from your own work.
- If required, run the approved participant protocol and insert only genuine de-identified results.
- Confirm final public, GitHub or OneDrive sharing links; no repository URL has been invented.
- Read every claim, code path, slide note and AI acknowledgement before submission.